COLLABORATIVE STUDY MATERIAL & QUIZ PLATFORM

Sprint 1 - Detailed Development, Testing, CI/CD and Project Tracking Plan

Course	UE24CS341A - Software Engineering
Team	Team 10
Member 1	Dharani S - Authentication + Study Material Management
Member 2	Prajwal M - Question Bank + Quiz Creation
Member 3	Karan Varshney - Quiz Attempt + Evaluation + Performance
Member 4	Harshit Chandak - Backend Foundation + Admin + Security + CI/CD

Prepared for Sprint 1 review - 02 October 2026

Aligned with the approved work division, SRS/SAD/Test Plan, and the faculty Sprint 1 instructions.

1. Sprint 1 Objective and Scope

Sprint 1 review target: Show working Sprint 1 features, unit tests, integration tests, meaningful Git/GitHub activity, updated Jira evidence, and a working CI/CD pipeline that automatically builds and runs the available tests on push/PR.

This plan expands only the work already assigned to Sprint 1. It does not pull later-sprint functionality forward merely to make the sprint look larger. The focus is to finish the agreed Sprint 1 features properly, test them, integrate them, and collect review evidence.

Member	Sprint 1 feature scope	Main review evidence
Member 1 - Dharani S	Registration, login, logout, User model/API, PDF upload	Working auth + upload PRs, unit/integration tests, screenshots/API evidence
Member 2 - Prajwal M	Question model/API, Add MCQ, question filtering	Question Bank PR + tests, filter demo, API contract evidence
Member 3 - Karan Varshney	QuizAttempt model, attempt UI prototype, evaluation logic prototype	Local working prototype + scoring/model tests + contract alignment
Member 4 - Harshit Chandak	Auth middleware, hashing, role foundations, Member 1 integration support, CI/CD	Security foundation PRs + green Jenkins pipeline + integration/security test evidence

1.1 Scope guardrails - what is NOT required in Sprint 1

Member	Keep for later sprint(s)
Member 1	Full material search/filter/view/download is Sprint 2. Sprint 1 only needs PDF upload plus auth flow.
Member 2	Full quiz creation/search/view is Sprint 2. Sprint 1 focuses on the Question Bank.
Member 3	Full submission/result/history/performance is Sprint 2/3. Sprint 1 is model + UI + evaluation prototype.
Member 4	Admin moderation, NFR load/performance testing, and final deployment/release are later. Sprint 1 is security foundation + integration + CI/CD.

1.2 Shared Sprint 1 Definition of Done

- Implementation is on a Jira-linked feature branch; no complete feature is developed directly on main.
- The feature has meaningful commits by the responsible member and a Pull Request is opened.
- At least one teammate reviews the PR; review comments are addressed before merge.
- Unit tests for the implemented logic pass and a coverage report is generated where the test runner supports code/branch coverage.
- Relevant integration tests across frontend/API/backend/MongoDB pass.
- The Jenkins pipeline is green for the branch/PR and automatically runs the configured build/test stages.
- Jira contains acceptance criteria, PR link, test evidence, screenshots/logs, and final status.
- No critical Sprint 1 defect remains open before the issue moves to DONE.

2. Recommended Sprint 1 Execution Sequence

Sequence	Team activity	Why it comes here
1	Lock Sprint 1 Jira issues, acceptance criteria, API contracts and branch naming.	Prevents duplicate work and late API changes.
2	Member 4 stabilizes auth/security middleware while Members 1-3 build their module code in parallel.	Member 1 depends directly on auth middleware; Member 3 needs shared auth conventions.
3	Each member writes unit tests alongside implementation; do not postpone test writing.	Faculty mail explicitly requires development and testing to progress together.
4	Run module integration tests and merge via PR to the integration/develop branch.	Finds frontend/API/database mismatches before review day.
5	Jenkins runs build/tests automatically; fix all red builds before continuing.	Review must demonstrate a working CI/CD pipeline.
6	Collect Jira/GitHub/test evidence and rehearse the Sprint 1 demo.	Makes individual contributions and traceability visible.

2.1 Key dependency map

Dependency	Required handoff
Member 4 -> Member 1	Auth middleware/token/session conventions, password hashing helper, protected-route behavior, common error format.
Member 2 -> Member 3	Question/quiz data contract. In Sprint 1, Member 3 may use seed/mock question data until full Quiz creation is ready in Sprint 2.
Member 4 -> All	MongoDB connection conventions, environment variables, shared middleware, integration branch rules, Jenkins pipeline.
All -> CI/CD	Each PR must include tests that can run non-interactively in Jenkins.

3. Member 1 - Dharani S

Sprint 1 ownership: Registration, login, logout, User model/API and PDF study-material upload.

3.1 Development work

- Create/finish Registration UI with name, email, password fields and client-side required/email validation.
- Implement the User model fields required for registration and authentication; keep password persistence compatible with Member 4 hashing middleware/service.
- Implement registration API integration and clear success/error feedback, including duplicate/invalid data handling.
- Create/finish Login UI and integrate login API; store the authenticated session/token using the team-agreed approach.
- Implement Logout so the session/token is cleared and protected UI routes are no longer accessible.
- Create the Material model fields required for Sprint 1 upload metadata: title, subject, description, file reference, uploader and timestamps as applicable.
- Build PDF upload UI with title, subject, description and file picker.
- Implement upload API integration and backend validation for authentication, required metadata, PDF-only file type and configured maximum file size.
- Return clear error messages for invalid upload cases instead of generic server errors.
- Coordinate with Member 4 on auth middleware and shared error handling; do not duplicate the shared security layer inside Member 1 code.

3.2 Suggested Sprint 1 Jira issues

Suggested issue	Acceptance target
S1-M1-01 Registration UI + API	Valid user can register; invalid/duplicate input is rejected clearly.
S1-M1-02 Login + logout flow	Valid login succeeds; invalid login fails; logout blocks protected access.
S1-M1-03 User model integration	User persists correctly and password is not stored in plain text.
S1-M1-04 PDF upload	Authenticated user can upload a valid PDF with metadata.
S1-M1-05 Upload validation	Non-PDF, oversized, missing-field and unauthenticated uploads are rejected.
S1-M1-06 Unit + integration tests	Sprint 1 auth/upload tests run in CI and evidence is attached to Jira.

3.3 Unit test cases to prepare

Test ID	Unit test	Expected result
UT-M1-01	Registration validation with valid details	Validation passes and service/controller can proceed.
UT-M1-02	Registration with missing/invalid email data	Validation fails with controlled error.
UT-M1-03	Login service/controller with valid credentials	Authentication succeeds and token/session is returned.
UT-M1-04	Login with incorrect credentials	Authentication is rejected; no protected data is returned.
UT-M1-05	PDF type validator	PDF accepted; non-PDF rejected.
UT-M1-06	File-size validator	Within-limit file accepted; oversized file rejected.

3.4 Integration test cases to prepare

Test ID	Integration flow	Expected result
IT-M1-01	Register -> login -> access protected endpoint	New user can authenticate and access protected functionality.

Test ID	Integration flow	Expected result
IT-M1-02	Login -> logout -> retry protected endpoint	Request is rejected after logout/session invalidation.
IT-M1-03	Authenticated upload -> MongoDB/material storage	Valid PDF metadata/file reference is stored and API returns success.
IT-M1-04	Unauthenticated upload request	Protected upload endpoint rejects the request.
IT-M1-05	Invalid file through frontend -> API	Error is handled consistently without corrupt database record.

3.5 GitHub, Jira and review evidence

- Use separate feature branches for auth and upload work; branch names should include the real Jira issue key.
- Open PRs rather than pushing completed features to main; request review from another member.
- Attach screenshots of Registration/Login/PDF Upload, API evidence (for example Postman), test output and PR links to Jira.
- Show the commit history under Member 1 identity so the implementation contribution is clearly visible.
- Before review, demonstrate: valid register/login/logout, invalid login, valid PDF upload, and rejected invalid upload.

Member 1 Sprint 1 completion evidence: Working auth/upload PRs + passing unit/integration tests + green CI build + Jira evidence. Material search/view/download can remain for Sprint 2.

4. Member 2 - Prajwal M

Sprint 1 ownership: Question model/API, Add MCQ and Question Bank filtering, plus Jira coordination quality.

4.1 Development work

- Create the Question schema/model with the fields required by the approved design: question text, options, correct answer, subject, topic and difficulty, plus timestamps/creator reference if the project uses them.
- Implement administrator-side Add Question UI and API integration.
- Validate required fields and ensure the MCQ representation has one valid correct answer tied to one of the provided options.
- Implement Question Bank list endpoint/UI for the Sprint 1 scope.
- Implement filtering by subject, topic and difficulty; verify combined filters behave predictably.
- Coordinate with Member 4 so question creation uses administrator authorization instead of relying on frontend-only restrictions.
- Define the question payload that Member 3 will consume. Student-facing question data must not expose correctAnswer before submission.
- Document the Question/Quiz API contract in a simple repository document or README section so Member 3 does not guess field names.
- As Jira coordinator, check that every Sprint 1 issue has an owner, acceptance criteria, priority, sprint assignment and visible dependency; members still update their own issues.

4.2 Suggested Sprint 1 Jira issues

Suggested issue	Acceptance target
S1-M2-01 Question schema/model	Required classification and MCQ fields persist correctly.
S1-M2-02 Add MCQ UI + API	Administrator can create a valid MCQ.
S1-M2-03 Question validation	Missing/invalid options or correct answer are rejected.
S1-M2-04 Question Bank filters	Subject/topic/difficulty filters return expected questions.
S1-M2-05 Student-safe payload	Student-facing response omits correctAnswer before submission.
S1-M2-06 API contract + Jira coordination	Member 3 has a stable contract; board contains owners, criteria and dependencies.

Suggested issue	Acceptance target
S1-M2-07 Unit + integration tests	Question Bank tests pass locally and in CI.

4.3 Unit test cases to prepare

Test ID	Unit test	Expected result
UT-M2-01	Create Question validation with valid MCQ	Validation passes; model is constructible.
UT-M2-02	MCQ missing question/options/subject	Validation rejects incomplete data.
UT-M2-03	Correct answer does not match allowed option	Validation rejects the MCQ.
UT-M2-04	Subject/topic/difficulty filter logic	Correct query/filter object is produced.
UT-M2-05	Student-safe serializer/DTO	correctAnswer is absent from student-facing payload.

4.4 Integration test cases to prepare

Test ID	Integration flow	Expected result
IT-M2-01	Admin auth -> Add Question API -> MongoDB	Valid question is stored and can be retrieved.
IT-M2-02	Student role -> Add Question API	Request is rejected by backend authorization.
IT-M2-03	Create several questions -> filter by subject/topic/difficulty	Only matching questions are returned.
IT-M2-04	Student-safe question endpoint	Question text/options returned without correctAnswer.
IT-M2-05	Question API contract consumed by Member 3 fixture/prototype	Field names/types are compatible; no manual remapping is required.

4.5 GitHub, Jira and review evidence

- Open Question Bank feature PR(s) with tests; request review from a teammate rather than self-merging without review.
- Maintain a clean Sprint 1 Jira board: TO DO -> IN PROGRESS -> CODE REVIEW -> TESTING -> DONE.
- Do not update other members' issues on their behalf; instead verify that required evidence is present before DONE.
- Attach Add MCQ screenshot, filter screenshot/API output, test log/coverage and PR links.
- Before review, demonstrate: valid MCQ creation, invalid MCQ rejection, filter behavior and student-safe response.

Member 2 Sprint 1 completion evidence: Question Bank PR + tests + working Add MCQ/filter demo + documented data contract. Full Quiz creation/search remains Sprint 2.

5. Member 3 - Karan Varshney

Sprint 1 ownership: QuizAttempt model + attempt UI prototype + evaluation logic prototype.

5.1 Development work

- Create the QuizAttempt/Result schema/model needed for later attempt persistence, including user reference, quiz reference, responses and/or score data, percentage and timestamp as required by the design.
- Build a Quiz Attempt UI prototype that can render question text and options and record selected answers.
- Because full Quiz creation is a Sprint 2 Member 2 deliverable, use agreed seed/mock question data in Sprint 1 where necessary rather than blocking the prototype.
- Implement a server-side or backend-oriented evaluation helper prototype that compares submitted responses with stored correct answers and calculates raw score/percentage.

- Handle unanswered questions predictably in the evaluation prototype.
- Agree request/response fields with Member 2 and Member 4; document them before implementing deeper integration.
- Keep the prototype compatible with Member 4 authentication middleware so the eventual attempt endpoint is protected.
- Do not spend Sprint 1 time completing history, subject-wise performance or the full result flow; those are later sprint deliverables.

5.2 Suggested Sprint 1 Jira issues

Suggested issue	Acceptance target
S1-M3-01 QuizAttempt schema/model	Attempt-related fields validate and can persist to test MongoDB.
S1-M3-02 Attempt UI prototype	Prototype displays MCQs and records selected answers.
S1-M3-03 Evaluation logic prototype	Known response sets produce correct score/percentage.
S1-M3-04 Question/attempt API contract	Fields are agreed with Members 2 and 4 and documented.
S1-M3-05 Unit + integration tests	Model/evaluation/protected-route prototype tests pass in CI.

5.3 Unit test cases to prepare

Test ID	Unit test	Expected result
UT-M3-01	Evaluation with all answers correct	Full score and 100% are produced.
UT-M3-02	Evaluation with all answers incorrect	Zero score is produced.
UT-M3-03	Evaluation with mixed correct/incorrect answers	Score exactly matches known expected result.
UT-M3-04	Evaluation with one or more unanswered questions	Unanswered items are handled consistently without crash.
UT-M3-05	QuizAttempt model required fields/timestamp	Valid attempt object passes; invalid structure is rejected.

5.4 Integration test cases to prepare

Test ID	Integration flow	Expected result
IT-M3-01	Authenticated prototype route -> QuizAttempt model -> test MongoDB	Test attempt data can be stored and read back.
IT-M3-02	Member 2 question payload/fixture -> Attempt UI	UI renders question/options without needing field-name changes.
IT-M3-03	Attempt UI selected answers -> evaluation helper/API prototype	Submitted response shape is accepted and produces expected score.
IT-M3-04	Unauthenticated access to protected attempt prototype	Request is rejected by shared auth middleware.

5.5 GitHub, Jira and review evidence

- Use a feature branch for QuizAttempt model/prototype and a PR reviewed by another member.
- Attach expected-vs-actual scoring evidence for the evaluation tests.
- Attach a screenshot/video of the attempt UI prototype using seed/mock questions if full quiz creation is not available yet.
- Attach the agreed API contract or repository documentation link in Jira.
- Before review, demonstrate: prototype loads questions, selection works, scoring helper passes known test cases, and attempt model can persist test data.

Member 3 Sprint 1 completion evidence: A local working attempt/evaluation prototype with a tested QuizAttempt model. It is acceptable for the prototype to use seed/mock questions because full quiz creation belongs to Sprint 2.

6. Member 4 - Harshit Chandak

Sprint 1 ownership: authentication/security foundation, role foundations, Member 1 integration support, GitHub integration health and CI/CD pipeline.

6.1 Development and security foundation work

- Confirm the shared Express application, MongoDB connection, environment configuration and common error-handling foundation are stable from Sprint 0.
- Implement/review password hashing using a suitable library (for example bcrypt) so no plain-text password is persisted.
- Implement/review JWT/session authentication middleware and protected-route handling.
- Implement role foundation middleware for administrator-only operations; Sprint 1 needs the foundation even though full moderation is later.
- Provide reusable request validation/error conventions so Members 1-3 do not invent incompatible patterns.
- Integrate Member 1 registration/login/logout and PDF upload with the shared auth middleware.
- Support Member 2 administrator-only Add Question route authorization.
- Support Member 3 protected attempt prototype and agreed auth/user context.
- Keep shared conventions in common middleware/utilities; do not take over Member 1-3 feature-specific schemas/controllers.

6.2 CI/CD pipeline work - mandatory Sprint 1 activity

- Set up Jenkins against the GitHub repository and configure it to trigger automatically on push/PR (webhook or equivalent supported setup).
- Pipeline stages should include: checkout, dependency installation, backend unit/integration tests, frontend tests (if present), project build, and coverage artifact generation where configured.
- Mark the pipeline failed when build/tests fail. The team should fix red builds before merging or starting dependent work.
- Keep secrets/credentials outside the repository using Jenkins credentials/environment configuration; do not commit .env secrets.
- Archive or retain useful test/coverage reports so Sprint 1 review can show objective evidence.
- If a development deployment target is already available, add a safe deploy-to-dev stage after successful tests; otherwise ensure the automatic build/test pipeline is fully working and keep final deployment for the later deployment sprint.
- Optionally wire static analysis (for example SonarQube or equivalent) if already available; this should not block the core Sprint 1 feature/test pipeline unless the faculty specifically requires it for the review.

6.3 Suggested Sprint 1 Jira issues

Suggested issue	Acceptance target
S1-M4-01 Password hashing foundation	No plain-text password storage; hashing/verification tests pass.
S1-M4-02 Authentication middleware	Valid auth accepted; missing/invalid auth rejected consistently.
S1-M4-03 Role authorization foundation	Admin route allows admin and rejects student role.
S1-M4-04 Member 1 integration support	Register/login/logout/upload work with shared auth conventions.
S1-M4-05 Jenkins CI/CD pipeline	Push/PR automatically triggers build + tests; green/red status is visible.
S1-M4-06 Security + integration tests	Auth/RBAC/invalid-input integration tests pass and evidence is attached.

6.4 Unit test cases to prepare

Test ID	Unit test	Expected result
UT-M4-01	Password hashing + verification	Hash differs from plain password and verification behaves correctly.

Test ID	Unit test	Expected result
UT-M4-02	Authentication middleware with valid token/session	Request proceeds with authenticated user context.
UT-M4-03	Authentication middleware with missing/invalid token/session	Request is rejected with safe error.
UT-M4-04	Role middleware: admin user	Administrator-only action is allowed.
UT-M4-05	Role middleware: student user	Administrator-only action is denied.
UT-M4-06	Common validation/error handling	Malformed input returns controlled response without sensitive details.

6.5 Integration and pipeline test cases

Test ID	Integration/pipeline flow	Expected result
IT-M4-01	Register/login from Member 1 -> shared auth middleware -> protected route	Authenticated flow works end to end.
IT-M4-02	Student role -> admin Add Question route	Backend returns authorization failure.
IT-M4-03	Valid/invalid auth -> PDF upload route	Valid auth allowed; invalid auth rejected.
CI-M4-01	Push a passing commit/PR	Jenkins triggers automatically and build/tests finish green.
CI-M4-02	Use a temporary intentionally failing test on a non-merge demo branch	Pipeline becomes red/failed, proving failures are detected; remove/fix before merge.

6.6 GitHub and review evidence

- Maintain repository structure, .gitignore, branch conventions and PR workflow; keep main stable.
- Help resolve integration conflicts without becoming the only committer or merger.
- Attach Jenkins pipeline file/configuration evidence, successful run screenshot, test/coverage artifacts and security test logs to Jira.
- Before review, demonstrate: hash/auth middleware, student-vs-admin authorization behavior, and a push/PR that triggers the Jenkins pipeline.

Member 4 Sprint 1 completion evidence: Security foundation PRs + successful integration with Member 1 + a demonstrably working Jenkins build/test pipeline. Full admin moderation, load testing and final deployment remain later sprint work.

7. Shared Sprint 1 Testing Plan

The Test Plan defines unit and integration testing as separate levels. Sprint 1 should therefore contain both module-level automated tests and a small but real set of cross-component tests across React/API/backend/MongoDB.

7.1 Minimum test execution set for Sprint 1 review

Area	Required demonstration
Authentication	Registration, valid login, invalid login, logout/protected access, password hashing.
PDF upload	Valid PDF upload, non-PDF rejection, oversized-file rejection, unauthenticated upload rejection.
Question Bank	Admin creates MCQ, invalid MCQ rejected, filter works, student cannot use admin create route.
QuizAttempt prototype	Question/options render, selected responses tracked, known scoring cases pass, model persists test data.
Security integration	Protected routes require auth; admin-only route rejects student; errors do not leak secrets.

Area	Required demonstration
CI/CD	A GitHub push/PR automatically triggers Jenkins, runs configured build/tests, and reports status.

7.2 Test case documentation format

- Test Case ID (use clear UT/IT/CI prefix).
- Module name and short test purpose.
- Preconditions.
- Detailed ordered test steps.
- Exact test data including valid and invalid values.
- Expected result.
- Actual result after execution.
- Pass/Fail status and defect/Jira reference when failed.

7.3 Coverage and evidence

- Generate code/branch coverage for automated unit tests where supported by the selected JavaScript test stack.
- Store or attach a coverage screenshot/report to the Sprint 1 test evidence.
- Do not chase an arbitrary coverage percentage if the faculty has not specified one; prioritize meaningful tests around Sprint 1 logic and error paths.
- Maintain a small Postman/API collection for integration smoke tests if the team is already using Postman.

8. Jira, GitHub and CI/CD Workflow for Every Member

Stage	What the member must do	Evidence
TO DO	Issue has clear description, owner, acceptance criteria, requirement link/dependency.	Jira issue ready in Sprint 1.
IN PROGRESS	Create Jira-linked feature branch and commit incrementally.	Branch + meaningful commits.
CODE REVIEW	Open PR and request teammate review.	PR, review comments/approval.
TESTING	Run unit + relevant integration tests; Jenkins must be green.	Test log/coverage + Jenkins run.
DONE	Merge approved code; attach PR/test/screenshots; no critical open defect.	Closed Jira issue with full traceability.

8.1 Recommended review pairing for Sprint 1

Developer	Suggested reviewer	Focus
Member 1	Member 2	Auth/upload behavior and interface clarity.
Member 2	Member 3	Question data contract and filter behavior.
Member 3	Member 4	Attempt model, evaluation logic and protected-route compatibility.
Member 4	Member 1 or Member 3	Auth/RBAC/security middleware and Jenkins changes.

9. Sprint 1 Review Checklist

Review rule: Do not present only screenshots. Be ready to show the running feature, relevant code, tests, GitHub PR/commit history, Jira issue evidence and the CI/CD run.

9.1 Team-level evidence to keep ready

- Jira Sprint 1 board showing owners, issue states, acceptance criteria and linked PR/test evidence.
- GitHub repository with individual branches/commits/PRs for all four members.
- At least one reviewed PR from each member or clear individual PR activity matching the work division.
- Jenkins job/pipeline screen showing automatic build/test execution from repository changes.
- Unit test report and code/branch coverage output for implemented Sprint 1 code.
- Integration test execution evidence for auth/upload/question bank/attempt prototype/security boundaries.
- List of defects found and fixed during Sprint 1, with Jira references where applicable.
- Short README section with how to set up/run/test the Sprint 1 integrated build.

9.2 Suggested 8-10 minute Sprint 1 demo order

Order	Demo item	Presenter
1	Jira board + GitHub branch/PR overview	Member 2 / Member 4
2	Registration, login, logout and PDF upload	Member 1
3	Add MCQ + Question Bank filtering + permission check	Member 2
4	Quiz Attempt UI/evaluation prototype using agreed test data	Member 3
5	Auth/RBAC foundation + Jenkins automatic build/test run	Member 4
6	Unit/integration test report + coverage + defects fixed	All members, 20-30 seconds each

9.3 Final Sprint 1 sign-off table

Check	Owner	Status before review
Member 1 auth + PDF upload implemented and tested	Member 1	To be marked Done
Member 2 Question Bank implemented and tested	Member 2	To be marked Done
Member 3 attempt/evaluation prototype implemented and tested	Member 3	To be marked Done
Member 4 auth/security foundation + CI/CD implemented and tested	Member 4	To be marked Done
All Sprint 1 PRs reviewed and merged to integration branch	All	To be marked Done
Jenkins green on integrated Sprint 1 build	Member 4 + All	To be marked Done
Jira evidence complete; no critical Sprint 1 defect open	All; Member 2 verifies board quality	To be marked Done

10. Source Alignment

This Sprint 1 plan is aligned to the project documents supplied in this chat and the faculty Sprint 1 mail. The detailed tasks above are a planning expansion of the already approved Sprint 1 ownership, not a change to the SRS or work division.

Source	Used for
Detailed Work Division for a 4-Member Software Engineering Team	Member ownership, Sprint 1 scope, Jira/GitHub workflow, review pairing and Definition of Done.
Software Requirements Specification v1.5	Functional roles, Student/Administrator boundaries, MERN architecture and protected operations.

Source	Used for
Software Architecture and Design Specification v1.0	Layered MERN component/API design and security/error-handling conventions.
Software Test Plan v1.0	Unit/integration test levels, Sprint 1 testing milestone and evidence expectations.
JACKFRUIT Software Engineering Project Guidelines	Git/GitHub, Jira, Jenkins CI/CD, PR/code review, unit test coverage and individual contribution expectations.
Faculty Sprint 1 mail	Concurrent development + unit/integration testing + CI/CD + project tracking + review demonstration requirements.

End of Sprint 1 Work Plan